ΠΑΝΕΠΙΣΤΗΜΙΟ ΙΩΑΝΝΙΝΩΝ

ΤΜΗΜΑ ΜΗΧ. Η/Υ & ΠΛΗΡΟΦΟΡΙΚΗΣ

ΕΠΕΞΕΡΓΑΣΙΑ ΦΥΣΙΚΗΣ ΓΛΩΣΣΑΣ

ΕΡΓΑΣΙΑ ΓΙΑ ΤΟ ΑΚΑΔΗΜΑΪΚΟ ΕΤΟΣ 2024-2025

ΟΜΑΔΑ NEURALHOLIC

ΙΩΑΝΝΗΣ ΦΙΛΛΗΣ, ΑΜ:5380

ΤΕΛΙΚΗ ΑΝΑΦΟΡΑ

ΙΟΥΝΙΟΣ 2025

1 ΠΕΡΙΕΧΟΜΕΝΑ

2 ΕΙΣΑΓΩΓΗ 3
3 ΔΗΜΙΟΥΡΓΙΑ ΑΡΝΗΤΙΚΩΝ ΠΑΡΑΔΕΙΓΜΑΤΩΝ 4
4 PREPROCESSING TEXT 4
4.1 LOWERCASE AND PUNCTUATION REMOVAL 4
4.2 STOP WORD REMOVAL 4
4.3 ΑΦΑΙΡΕΣΗ ΤΩΝ ΠΙΟ ΣΥΧΝΩΝ ΛΕΞΕΩΝ 5
5 CREATION OF TEXT FEATURES 7
5.1 TF-IDF 7
5.2 WORD2VEC 7
5.3 DOC2VEC 8
5.4 SENTENCE TRANSFORMER 8
6 AUTHOR OVERLAP 8
7 GRAPH FEATURES 8
7.1 NODE2VEC 9
7.2 ΕΠΙΠΛΕΟΝ ΠΛΗΡΟΦΟΡΙΑ ΓΡΑΦΗΜΑΤΟΣ 9
7.2.1 IN DEGREE/OUT DEGREE 9
7.2.2 PAGERANK 9
7.2.3 NUMBER OF COMMON NEIGHBORS-V1 9
7.2.4 NUMBER OF COMMON NEIGHBORS-V2 9
7.3 KATZ CENTRALITY 10
7.4 EIGENVECTOR CENTRALITY 10
7.5 HITS 10
7.6 ENGINEERED FEATURES 10
8 MODELS 11
8.1 GAUSSIAN NAIVE BAYES 11
8.2 LOGISTIC REGRESSION 14
8.3 SGD CLASSIFIER 15
8.4 DECISION TREE CLASSIFIER 16
8.5 RANDOM FOREST CLASSIFIER 17
8.6 MLP CLASSIFIER 18
8.7 XGBOOST 18
8.8 K-NN CLASSIFIER 19
9 ΣΥΝΟΨΗ ΚΑΙ ΣΥΜΠΕΡΑΣΜΑΤΑ 20

2 ΕΙΣΑΓΩΓΗ

Η παρούσα αναφορά συγγράφεται στα πλαίσια της εργασίας του μαθήματος Επεξεργασία Φυσικής Γλώσσας από τον φοιτητή Γιάννη Φίλλη (ΑΜ:5380) κατά το εαρινό εξάμηνο του ακαδημαϊκού έτους 2024-2025.

Η εργασία του μαθήματος είχε ως θέμα την πρόβλεψη για τον αν ένα paper αναφέρει (cite) κάποιο άλλο. Στο πλαίσιο αυτό, μας δόθηκε αρχικά το αρχείο "abstracts.txt" όπου είχε τα paper id και τα αντίστοιχα abstracts. Επίσης μας δόθηκε το αρχείο "authors.txt" το οποίο περιείχε τα paper id και την λίστα των συγγραφέων του κάθε paper. Ένα ακόμα αρχείο που μας δόθηκε ήταν το "edgelist.txt" το οποίο περιέχει πληροφορία για τις ακμές του γραφήματος. Τέλος, μας δόθηκε και το "test.txt" το οποίο είναι το αρχείο από το οποίο παίρνουμε τα αποτελέσματα και τα κάνουμε submit στο Kaggle.

Για να αντιμετωπίσω αυτό το πρόβλημα, εφάρμοσα πολλές από τις τεχνικές που συζητήθηκαν στο μάθημα. Ποιο συγκεκριμένα, όσον αφορά την πληροφορία που μπορούμε να εξάγουμε από τα κείμενα, αρχικά αφαίρεσα τις "stop words" και αποφάσισα να αφαιρέσω και κάποιες από τις πιο κοινές λέξεις που συναντάμε στα κείμενα καθώς αυτές δεν προσδίδουν πολύ πληροφορία. Έπειτα εφάρμοσα την τεχνική Tf-idf. Ακόμα, θέλοντας να εκμεταλλευτώ την χρησιμότητα των Embeddings εφάρμοσα τις τεχνικές word2vec και doc2vec. Τέλος, χρησιμοποίησα το Sentence Transformer, ενώ για τους συγγραφείς χρησιμοποίησα την τεχνική Sorensen-Dice ώστε να βρω την ομοιότητα που έχουν οι λίστες των συγγραφέων από δύο papers.

Για την εξαγωγή πληροφορίας από το γράφημα, εφάρμοσα τεχνικές όπως node2vec, katz centrality και eigenvector centrality. Ακόμα, χρησιμοποιήθηκαν και άλλες πληροφορίες που μπορούν να εξαχθούν από το γράφημα και θα αναλυθούν λεπτομερώς στην συνέχεια.

Τέλος, χρησιμοποίησα αρκετά μοντέλα για τις προβλέψεις, όπως ο Bayes, Logistic Regression, MLP, XGBOOST κτλ.

Όπως ζητήθηκε, στα παραδοτέα είναι η παρούσα αναφορά, το powerpoint της παρουσίασης και ο φάκελος με τον κώδικα καθώς και τα training, test set και το αρχείο results το οποίο περιέχει τα αποτελέσματα του Feature Importance από το αρχείο του κώδικα και τα αποτελέσματα που αναφέρονται στην ενότητα με τα Engineered Features. Το training set προφανώς λόγο του μεγάλου όγκου του περιέχει μόνο τα ζεύγη των papers και το label.

3 ΔΗΜΙΟΥΡΓΙΑ ΑΡΝΗΤΙΚΩΝ ΠΑΡΑΔΕΙΓΜΑΤΩΝ

Από το αρχείο ***edgelist.txt*** παίρνουμε όλες τις γνωστές συνδέσεις των papers, ωστόσο αυτό δεν είναι αρκετό για την εκπαίδευσή μας. Χρειάζεται να έχουμε και ζεύγη paper, όπου γνωρίζουμε πως δεν συνδέονται μεταξύ τους.

Έτσι λοιπόν, αυτό που έκανα ήταν ότι δημιούργησα τον κώδικα προκειμένου να παράγω τα negative examples. Πιο συγκεκριμένα, αυτό που κάνω είναι ότι επιλέγω τυχαία δύο paper id και έπειτα για την δυάδα (i,j) ελέγχω πως δεν υπάρχει ούτε η (i,j) ούτε η (j,i) σε αυτά που έχω επιλέξει.

Αυτό είναι πολύ σημαντικό, καθώς θέλουμε να αποφύγουμε και την περίπτωση δύο ή περισσότερων ίδιων ζευγαριών αλλά και την περίπτωση να υπάρχει το ζευγάρι ως ακμή (δηλαδή το (i,j) να συνδέεται) και εμείς να προσθέσουμε το (j,i) ότι δεν έχει σύνδεση, καθώς έτσι το μοντέλο μας θα μπερδευτεί.

4 PREPROCESSING TEXT

Ένα πολύ σημαντικό κομμάτι της όλης διαδικασίας είναι το κομμάτι της επεξεργασίας του κειμένου. Οι περισσότερες τεχνικές απαιτούν να γίνει ένα καλό preproccesing πριν δώσουμε τα κείμενα για μάθηση. Όπως γνωρίζουμε και από την θεωρία, υπάρχουν αρκετές τεχνικές οι οποίες μπορούν να εφαρμοστούν προκειμένου να επεξεργαστούμε κάποιο κείμενο.

Έτσι λοιπόν, δημιουργώ μία συνάρτηση η οποία ονομάζεται ***preprocess_abstract()*** και η οποία φροντίζει να εφαρμόσει τις ακόλουθες τεχνικές.

4.1 LOWERCASE AND PUNCTUATION REMOVAL

Αρχικά, το πρώτο που θα κάνουμε είναι να μετατρέψουμε όλους τους χαρακτήρες σε μικρούς, προκείμενου να μην έχουμε περίπτωση όπου για παράδειγμα έχουμε τις λέξεις Kaggle και kaggle όπου ενώ είναι οι ίδιες, θα θεωρούνται διαφορετικές επειδή η μία έχει κεφαλαίο γράμμα. Έπειτα, θα αφαιρέσουμε όλα τα σημεία στίξης καθώς δεν μας προσφέρουν κάποια πληροφορία.

4.2 STOP WORD REMOVAL

Η τελευταία τεχνική που εφάρμοσα είναι να αφαιρέσω της stop words από τα κείμενα, δηλαδή τις λέξεις οι οποίες εμφανίζονται πολύ συχνά στα κείμενα και δεν μας προσφέρει κάποια πληροφορία. Χαρακτηριστικά παραδείγματα stop words είναι the, and, to.

4.3 ΑΦΑΙΡΕΣΗ ΤΩΝ ΠΙΟ ΣΥΧΝΩΝ ΛΕΞΕΩΝ

Ένα επίσης σημαντικό κομμάτι της προεπεξεργασίας των κειμένων είναι η αφαίρεση των πιο συχνών λέξεων, καθώς αυτές δεν μας προσφέρουν τόσο πολύτιμη πληροφορία από την στιγμή που εμφανίζεται σε πάρα πολλά κείμενα, ακόμα και σε αυτά που δεν σχετίζονται μεταξύ τους. Πρέπει ωστόσο να είμαστε προσεκτικοί ώστε να μην αφαιρέσουμε λέξεις οι οποίες αναφέρονται σε συγκεκριμένα topics.

Χρησιμοποιώντας από την βιβλιοθήκη το Counter, μπορούμε να έχουμε μία εικόνα για το τι γίνεται με τις λέξεις.

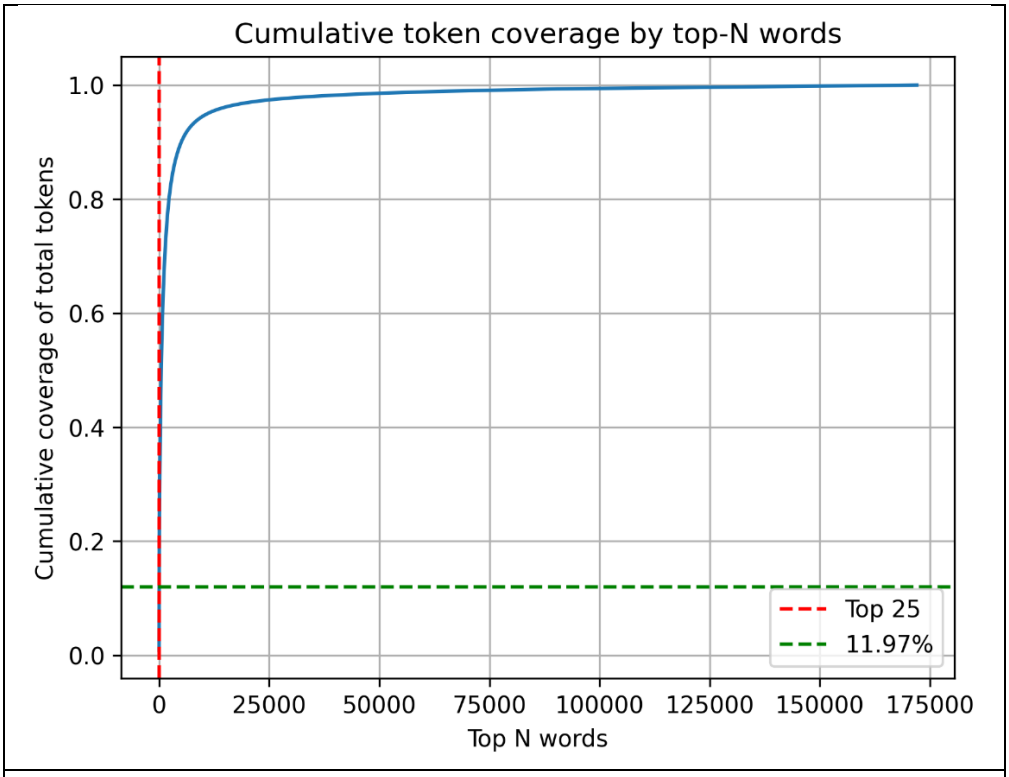
```
Top 25 most common words:
data: 108049
paper: 81390
method: 80836
model: 79453
proposed: 73794
algorithm: 67693
results: 67244
image: 63598
based: 62924
learning: 61864
using: 61687
approach: 59583
problem: 57377
show: 54963
methods: 50361
propose: 47718
information: 47205
performance: 45265
two: 43568
new: 42514
images: 40521
used: 38388
different: 37846
features: 37813
also: 36922
```

Εικόνα 4.1: 25 most common words

Όπως φαίνεται και στην παραπάνω εικόνα, οι 25 πιο συχνές λέξεις είναι αρκετά γενικές, εμφανίζονται πάρα πολύ συχνά και δεν φαίνεται πως μπορούν να προσφέρουν και πολλά στις τεχνικές μας.

Στην παρακάτω εικόνα, αυτό που μπορούμε να δούμε είναι η γραφική παράσταση που αντιπροσωπεύει τις εμφανίσεις των tokens (λέξεων) προς τις συνολικές εμφανίσεις.

Μπορούμε να παρατηρήσουμε πως οι 25 πιο συχνές λέξεις αντιστοιχούν περίπου στο 12% των συνολικών εμφανίσεων. Οπότε, μπορούμε να πούμε πως είναι ένας καλός αριθμός λέξεων που μπορούμε να αφαιρέσουμε.



Εικόνα 4.2: cumulative coverage

Αν θέλουμε να δούμε αυτές τις λέξεις και με έναν διαφορετικό τρόπο, παρακάτω ακολουθεί μία εικόνα που περιέχει λέξεις που εμφανίζονται και το μέγεθός τους αντιπροσωπεύει το πλήθος των εμφανίσεών τους.

Εικόνα 4.3: word cloud

Έτσι λοιπόν, έχουμε πειστεί ότι είναι ένας καλός αριθμός λέξεων να αφαιρέσουμε από τα κείμενά μας.

5 CREATION OF TEXT FEATURES

Αφού λοιπόν ολοκληρώσαμε την προεπεξεργασία των κειμένων, αυτό που θα κάνουμε τώρα είναι να εφαρμόσουμε τις γνωστές τεχνικές.

5.1 TF-IDF

Η πρώτη τεχνική που θα εφαρμόσουμε είναι η τεχνική του ***TF-IDF***. Η συγκεκριμένη τεχνική λαμβάνει υπόψιν πόσο συχνά μία λέξη εμφανίζεται σε ένα κείμενο, καθώς και πόσο σπάνια είναι σε όλα τα διαθέσιμα κείμενα.

Έτσι, εφαρμόζω το ***TF-IDF*** χρησιμοποιώντας την βιβλιοθήκη ***sklearn***. Ένα σημαντικό πρόβλημα που εμφανίζεται, είναι ότι υπάρχουν πολλά papers τα οποία έχουν κενά abstracts. Κάτι τέτοιο μπορεί να είναι πολύ κακό για το μοντέλο μας καθώς όταν θα πάμε να υπολογίσουμε την ομοιότητα κειμένων, κάθε φορά με αυτά η ομοιότητα θα βγαίνει μηδέν, καθώς δεν έχει καθόλου κείμενο.

Για να λύσω αυτό το πρόβλημα, αυτό που επέλεξα να κάνω είναι να δημιουργήσω το γράφημα (χρησιμοποιώντας την βιβλιοθήκη ***networkX***). Έπειτα δημιουργώ μία συνάρτηση, η οποία παίρνει σαν όρισμα ένα paper id. Έπειτα υπολογίζει τους γείτονες του συγκεκριμένου paper, παίρνει τα tf-idf διανύσματα που υπολογίστηκαν νωρίτερα και υπολογίζει τον μέσο όρο. Σε περίπτωση που δεν έχει κάποιο γείτονα, τότε απλά παίρνει το μηδενικό διάνυσμα. Αυτή την συνάρτηση θα την εφαρμόσουμε σε όλα τα paper που δεν έχουν abstract.

Αφού λοιπόν υπολογίσουμε και αυτά, το μόνο που μένει να κάνουμε είναι να υπολογίσουμε την ομοιότητα των ζευγών, χρησιμοποιώντας την ***cosine similarity*** τεχνική, τόσο στο training set, όσο και στο testing set.

5.2 WORD2VEC

Η επόμενη τεχνική που θα χρησιμοποιήσουμε είναι η τεχνική ***word2vec***. Αυτό που κάνει είναι να δημιουργεί embeddings, δηλαδή διανύσματα, τα οποία αναπαριστούν τις λέξεις και έχει την δυνατότητα να λαμβάνει υπόψιν και την σημασιολογία της λέξης.

Έτσι, χρησιμοποιώντας την βιβλιοθήκη ***gensim***, δημιουργώ το μοντέλο μου. Θέτω ως μέγεθος του διανύσματος 400, window_size ίσο με 5, ελάχιστες λέξεις το 2, το εκπαιδεύω με 20 εποχές και χρησιμοποιώ την αρχιτεκτονική ***Skip-Gram.*** Ο λόγος που επέλεξα την συγκεκριμένη αρχιτεκτονική είναι επειδή αυτό που κάνει είναι ότι λαμβάνοντας υπόψιν μία λέξη, προσπαθεί να προβλέψει τις γύρω του, κάτι το οποίο μου φάνηκε λίγο καλύτερο όσον αφορά τα επιστημονικά κείμενα.

Έπειτα, δημιουργήσαμε μία συνάρτηση, η οποία το μόνο που κάνει είναι να παίρνει τα embeddings του κάθε paper. Όπως και πριν, έχουμε το πρόβλημα με τα κενά abstracts και ο τρόπος με τον οποίο θα το λύσουμε είναι παρόμοιος, δηλαδή θα πάρουμε το μέσο όρο από τους γείτονες.

Τέλος, για να υπολογίσουμε την ομοιότητα των ζευγών, θα χρησιμοποιήσουμε πάλι την ***cosine similarity*** τεχνική.

5.3 DOC2VEC

Η επόμενη τεχνική ονομάζεται ***Doc2vec***. Πρόκειται για μία επέκταση της ***word2vec*** όπου εδώ έχουμε διανυσματική αναπαράσταση ολόκληρου του κειμένου και όχι της κάθε λέξης. Επομένως, χρησιμοποιώντας πάλι την βιβλιοθήκη ***gensim*** αρχικά αναθέτουμε tags στα κείμενα. Έπειτα δημιουργούμε το μοντέλο μας με vector_size ίσο με 400, window ίσο με 10, min_count ίσο με 2 και το εκπαιδεύουμε με 20 εποχές.

Αμέσως μετά, αυτό που κάνουμε είναι να δημιουργήσουμε ένα dictionary με κλειδί το paper id και value το embedding. Εφόσον έχουμε το ίδιο πρόβλημα με πριν, όπου έχουμε κενά κείμενα, αυτό που θα κάνουμε είναι πάλι να πάρουμε τον μέσο όρο από τους γείτονες.

Ολοκληρώνοντας, θα χρησιμοποιήσουμε πάλι την ***cosine similarity*** για να εξάγουμε την ομοιότητα μεταξύ των ζευγών.

5.4 SENTENCE TRANSFORMER

Η επόμενη τεχνική που χρησιμοποίησα είναι η ***sentence transformer.*** Χρησιμοποιώντας λοιπόν την ομώνυμη βιβλιοθήκη, περνώ σαν παράμετρο το ***allenai-specter*** το οποίο είναι ένα pretrained model ειδικό για επιστημονικά κείμενα. Οπότε με αυτό σε σχέση με άλλα μοντέλα, αναμένουμε να έχουμε καλύτερη επίδοση.

Οπότε, χρησιμοποιώντας την ***.encode()*** δημιουργούμε τα embeddings για τα κείμενά μας. Φυσικά, όπως και πριν χρησιμοποιούμε τους γείτονες, προκειμένου να πάρουμε τον μέσο όρο για τα κείμενα που είναι κενά. Χρησιμοποιώντας ξανά την ***cosine similarity*** υπολογίζουμε την ομοιότητα των ζευγών.

6 AUTHOR OVERLAP

Προκειμένου να υπολογίσω την ομοιότητα των συγγραφέων επέλεξα να χρησιμοποιήσω την τεχνική Sorensen-Dice. Αν και αρχικά μία ακόμα υποψήφια τεχνική ήταν η jacard, επέλεξα τελικά την άλλη καθώς στην συγκεκριμένη τεχνική, πρακτικά αυτό που συμβαίνει είναι ότι δίνει περισσότερο έμφαση στο αν υπάρχουν κοινοί συγγραφείς παρά στο αν δεν υπάρχουν, δηλαδή δίνεται περισσότερο έμφαση στο πόσοι κοινοί είναι ως προς το συνολικό πλήθος των συγγραφέων παρά στο να τα διαιρεί με όλους τους μοναδικούς συγγραφείς.

7 GRAPH FEATURES

Ένα ακόμα σημαντικό κομμάτι από το οποίο μπορούμε να εξάγουμε χρήσιμη πληροφορία είναι το κομμάτι του γραφήματος. Με βάση το αρχείο ***edgelist.txt*** που μας δόθηκε μπορούμε να δημιουργήσουμε και να εκμεταλλευτούμε το γράφημα που δημιουργείται με βάση τις σχέσεις των papers.

7.1 NODE2VEC

Θα ξεκινήσουμε με την τεχνική ***node2vec***. Πρόκειται για μία τεχνική παρόμοια με το ***word2vec*** και ***doc2vec*** καθώς και εδώ έχουμε παραγωγή διανυσμάτων, ωστόσο εδώ αναπαριστούμε πληροφορία για τον κόμβο στο γράφημα.

Έτσι το πρώτο που κάνουμε είναι να εκτελέσουμε στο τερματικό την εντολή ***pip install node2vec*** προκειμένου να εγκαταστήσουμε την βιβλιοθήκη που θα χρειαστούμε. Έπειτα με την χρήση της βιβλιοθήκης networkX δημιουργούμε το γράφημά μας. Το επόμενο που θα κάνουμε είναι να αρχικοποιήσουμε το μοντέλο. Χρησιμοποιώ dimensions = 512, walk_length=30, num_walks=100. Αμέσως μετά θα καλέσουμε την συνάρτηση ***.fit()*** με window=10, min_count=2 και batch_word = 4. Αφού ολοκληρώσουμε και με αυτό, το μόνο που μένει είναι να υπολογίσουμε την ομοιότητα, χρησιμοποιώντας και πάλι την μέθοδο ***cosine similarity.***

7.2 ΕΠΙΠΛΕΟΝ ΠΛΗΡΟΦΟΡΙΑ ΓΡΑΦΗΜΑΤΟΣ

Εκτός από τα embeddings, μπορούμε να εξάγουμε πληροφορία με βάση τους γειτονικούς κόμβους στο γράφημα.

7.2.1 IN DEGREE/OUT DEGREE

Οπότε, το πρώτο που θα χρησιμοποιήσουμε είναι το ***in degree*** για τους κόμβους οι οποίοι είναι τα cited_papers και αναπαριστά το πλήθος των κόμβων που «δείχνουν» σε αυτόν και το ***out degree*** για τους κόμβους που είναι τα citing papers και δείχνει το πλήθος των κόμβων στους οποίους «δείχνει». Σκέφτηκα πως μία τέτοια πληροφορία θα είναι χρήσιμη καθώς αν ο κόμβος που είναι citing paper δείχνει σε πολλά papers τότε είναι πιθανόν να δείχνει και στο συγκεκριμένο που θέλουμε. Παρόμοια η λογική και για το in degree.

7.2.2 PAGERANK

Το επόμενο που χρησιμοποίησα είναι το ***pagerank***. Το σκεπτικό πίσω από αυτή την τεχνική είναι ότι ένας κόμβος είναι σημαντικός αν «δείχνουν» σε αυτόν πολλοί σημαντικοί κόμβοι. Έτσι πρόσθεσα και για το citing paper και για το cited paper.

7.2.3 NUMBER OF COMMON NEIGHBORS-V1

Το επόμενο που έκανα είναι ότι υπολόγισα με βάση την jaccard, τους κοινούς κόμβους που δείχνουν και στο κόμβο i, και στον j. Το σκεπτικό πίσω από αυτό είναι ότι είναι πιο πιθανό ο i να δείχνει στον j αν έχουν πολλά κοινά papers που δείχνουν σε αυτά.

7.2.4 NUMBER OF COMMON NEIGHBORS-V2

Το επόμενο που χρησιμοποίησα είναι ότι πήρα τους κόμβους που δείχνει ο i, τους κόμβους που δείχνουν στον j και έπειτα υπολόγισα τους κοινούς τους κόμβους.

Σκέφτηκα πως αν υπάρχουν πολλοί κοινοί κόμβοι αυτής της κατηγορίας τότε είναι πιο πιθανό ο i να δείχνει στον j.

7.3 KATZ CENTRALITY

Μία ακόμα τεχνική που χρησιμοποίησα είναι η ***katz centrality***. Η συγκεκριμένη, αυτό που μετράει είναι η σημαντικότητα του κόμβου μετρώντας όλα τα μονοπάτια. Την εφάρμοσα τόσο για το citing paper όσο και για το cited paper.

7.4 EIGENVECTOR CENTRALITY

Άλλη μία τεχνική που χρησιμοποίησα είναι η ***eigenvector centrality***. Είναι παρόμοια με την katz centrality, ωστόσο τώρα μετράμε πόσο σημαντικός είναι ένας κόμβος λαμβάνοντας υπόψιν το πόσο σημαντικοί είναι και οι γείτονές του. Ομοίως, το εφάρμοσα και στο citing paper και στο cited paper.

Ωστόσο, η συγκεκριμένη τεχνική έχει ως αρνητικό ότι δεν έχει τόσο καλή επίδοση εάν το γράφημα δεν είναι καλά συνδεδεμένο.

7.5 HITS

Ακόμα, αποφάσισα να χρησιμοποιήσω το ***Hits*** που χρησιμοποιείται σε γραφήματα. Αυτό, έχει δύο δείκτες: το ***authority*** και το ***hub***. Πρακτικά, αυτό που γίνεται είναι ότι αν το authority έχει μεγάλη τιμή, τότε «σημαντικά» papers (κόμβοι) δείχνουν σε αυτό ενώ αν το hub έχει μεγάλη τιμή σημαίνει ότι αναφέρεται σε πολλά «σημαντικά» papers (κόμβοι).

7.6 ENGINEERED FEATURES

Αποφάσισα επίσης να χρησιμοποιήσω και κάποια engineered features. Για να καταλάβω ποια είναι σημαντικά, χρησιμοποίησα την mutual_info_classif με την βοήθεια της οποίας υπολόγισα το gain που έχω αν χρησιμοποιήσω το γινόμενο συνδυασμών από τα features μου. Αυτό που υπολογίζω είναι το πόσο χρήσιμο είναι το feature μόνο του, καθώς και πόσο χρήσιμο είναι το γινόμενό του με κάθε άλλο feature. Παρακάτω παραθέτω τα πιο χρήσιμα γινόμενα.

Top 10 interaction meta-features by MI gain:

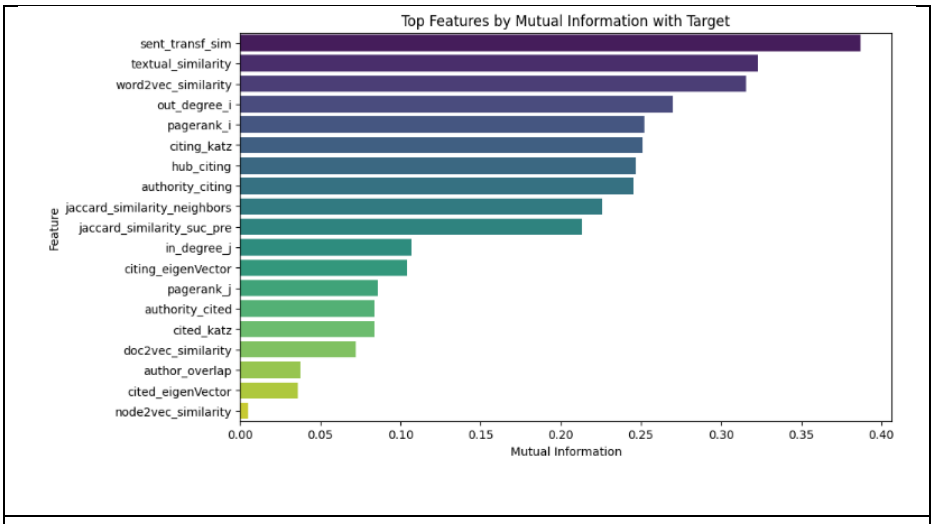
64]:

	feature1	feature2	mi1	mi2	mi_interaction	interaction_gain
2	textual_similarity	out_degree_i	0.322971	0.270089	0.398421	0.075449
37	in_degree_j	out_degree_i	0.106728	0.270089	0.314506	0.044417
44	in_degree_j	doc2vec_similarity	0.106728	0.072322	0.150259	0.043531
55	out_degree_i	pagerank_j	0.270089	0.085823	0.310073	0.039984
60	out_degree_i	doc2vec_similarity	0.270089	0.072322	0.306985	0.036896
52	in_degree_j	hub_citing	0.106728	0.246955	0.265458	0.018503
130	word2vec_similarity	sent_transf_sim	0.315639	0.387235	0.399866	0.012631
7	textual_similarity	word2vec_similarity	0.322971	0.315639	0.331888	0.008916
1	textual_similarity	in_degree_j	0.322971	0.106728	0.327071	0.004100
58	out_degree_i	word2vec_similarity	0.270089	0.315639	0.318963	0.003323

Εικόνα 7.1: Top 10 interaction features

Αυτά θα μας φανούν αρκετά χρήσιμα στην συνέχεια καθώς θα μπορούμε να τα χρησιμοποιήσουμε στα μοντέλα που θα εφαρμόσουμε.

Στο παρακάτω γράφημα βλέπουμε και τα features και πόσο σημαντικά είναι με βάση το label (και μετρική πάλι το gain).



Εικόνα 7.2: Top features

8 MODELS

Εφόσον λοιπόν τελειώσαμε με όλα αυτά, αυτό που μένει τώρα είναι να επιλέξουμε τα καλύτερα features και να τα εφαρμόσουμε στα μοντέλα μας. Ως μετρική χρησιμοποιώ την log_loss η οποία χρησιμοποιείται και στον διαγωνισμό του Kaggle. Προκειμένου να αξιολογήσω τα μοντέλα μου τοπικά, χρησιμοποίησα την cross-val-predict από την βιβλιοθήκη sklearn ώστε να παίρνω μία εκτίμηση για την επίδοσή τους.

Ακόμα, σε πολλά από τα επόμενα μοντέλα χρησιμοποίησα το ***RFEC***, το οποίο αυτό που κάνει είναι ότι δοκιμάζει σε κάθε μοντέλο όλους τους συνδυασμούς των features και κρατάει αυτά τα οποία είναι καλύτερα.

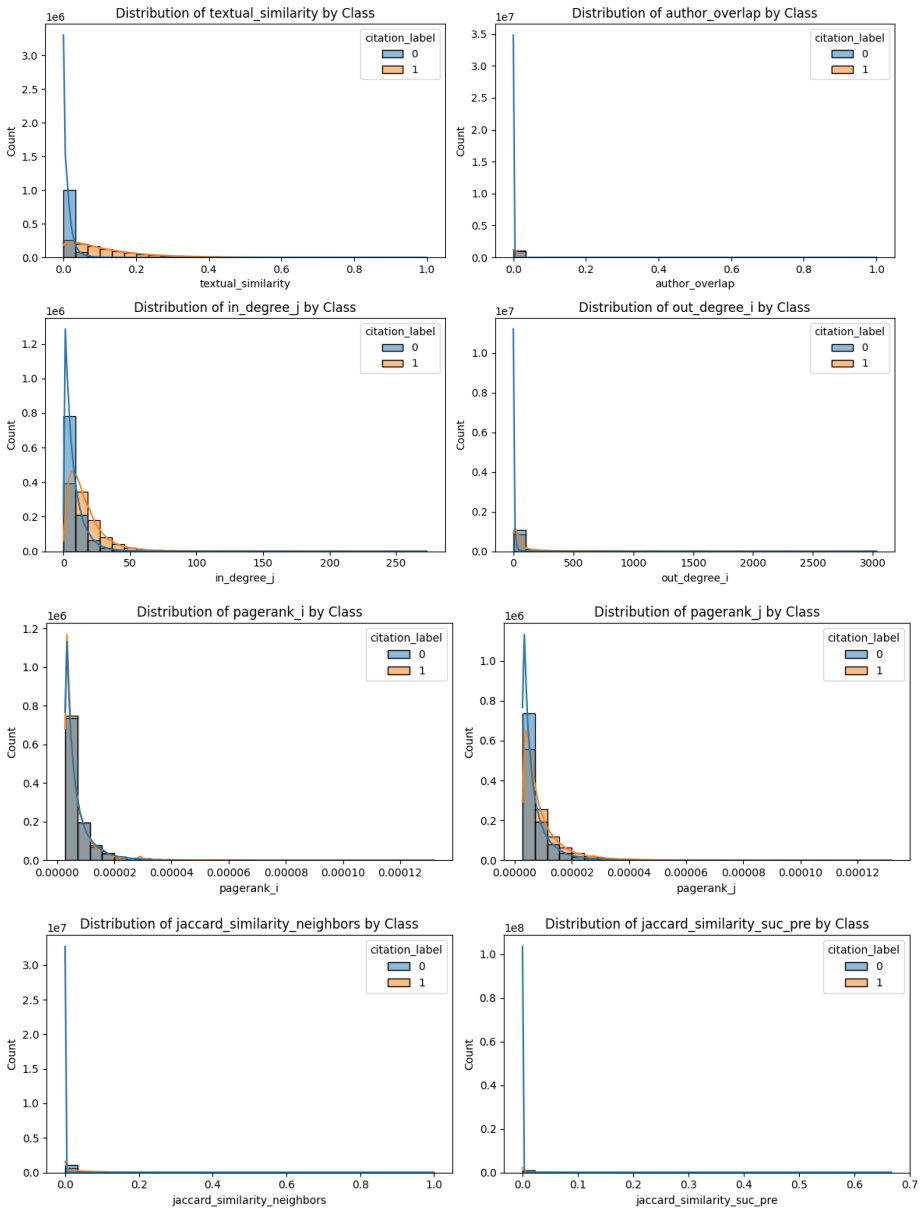
8.1 GAUSSIAN NAIVE BAYES

Ένα από τα πρώτα μοντέλα που συζητήσαμε και μέσα στο μάθημα είναι το μοντέλο naïve bayes. Καθώς έχουμε συνεχείς τιμές, θα χρησιμοποιήσουμε την gaussian, αφού όλες οι υπόλοιπες κατηγορίες δουλεύουν με διακριτά δεδομένα. Επίσης θα χρησιμοποιήσουμε και την minMax scaler καθώς για τέτοια μοντέλα είναι σχεδόν απαραίτητο.

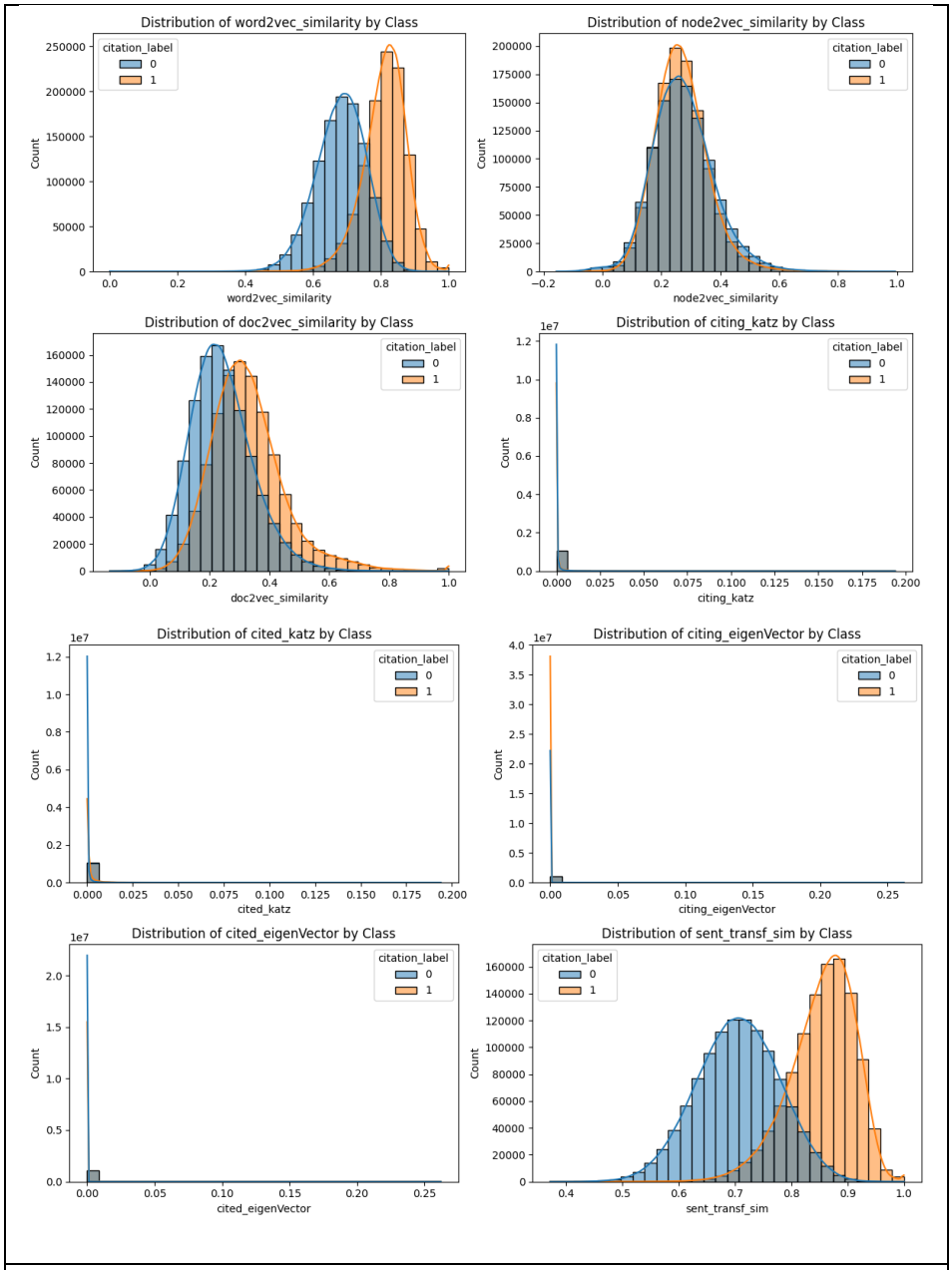
Για να δοκιμάσω στην αρχή το μοντέλο και να δω τι επιδόσεις έχει, αρχικά έβαλα το tf-idf similarity, το author overlap και έπειτα το word2vec. Η επιδόσεις(με το cross-validation) ήταν:

Features	Gaussian NB
Tf-idf sim, author overlap	0.92709
Tf-idf sim, author overlap, word2vec	0.82102

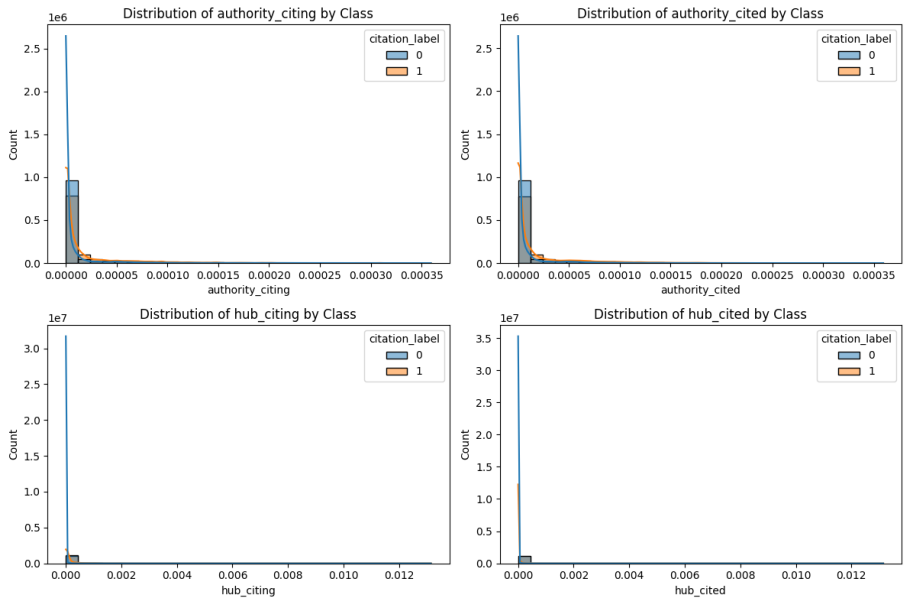
Τα αποτελέσματα δεν είναι και πάρα πολύ καλά. Αυτό οφείλεται στο γεγονός ότι στην Gaussian NB, πρέπει τα δεδομένα που χρησιμοποιούνται ως features να ακολουθούν την gaussian κατανομή. Έτσι αυτό που θα κάνω είναι να μελετήσω τις κατανομές των features.



Εικόνα 8.1: Κατανομές των Features-v1



Εικόνα 8.2: Κατανομές των Features-v2



Εικόνα 8.3: Κατανομές των Features-v3

Από τα παραπάνω μπορούμε να παρατηρήσουμε πως ακολουθούν, ή καλύτερα προσεγγίζουν, την gaussian κατανομή τα ***word2vec***, ***node2vec***, ***doc2vec*** και ***sent_transf_sim***. Δοκιμάζοντας διάφορους συνδυασμούς αυτών έχουμε.

Features	Gaussian NB
node2vec, word2vec, doc2vec, sent_transf	0.35133
node2vec, word2vec	0.39071
node2vec , doc2vec	0.63474
node2vec, sent_transf	***0.31508***
node2vec, word2vec, sent_transf	0.34055

Από τα παραπάνω, παρατηρούμε ότι την καλύτερη επίδοση φαίνεται να την έχει το μοντέλο μας όταν χρησιμοποιούμε τα ***node2vec***, ***sent_transf***. Έτσι, αν το κάνουμε submit στο Kaggle, παίρνουμε μία βαθμολογία που ισούται με ***0.34454***, λίγο υψηλότερη από αυτό που έχουμε τοπικά.

8.2 LOGISTIC REGRESSION

Το επόμενο μοντέλο που θα χρησιμοποιήσουμε είναι το ***Logistic Regression***. Εδώ αρχικά θα χρησιμοποιήσουμε τις default τιμές του classifier. Ως features θα χρησιμοποιήσουμε πάλι για αρχή τα tf-idf similarity, το author overlap και έπειτα το word2vec.

Features	Logistic Regression
Tf-idf sim, author overlap	0.3689
Tf-idf sim, author overlap, word2vec	0.3015

Για να πάρουμε τα πιο χρήσιμα features, θα χρησιμοποιήσουμε την πληροφορία από την εικόνα 6.2 όπου έχουμε μία κατάταξη των features με βάση την μετρική gain. Από αυτά, θα επιλέξω να χρησιμοποιήσω τα 10 πρώτα. Θα χρησιμοποιήσω επίσης και το ***RFEC*** το οποίο θα δοκιμάσει όλα τα features και θα επιλέξει τα καλύτερα και θα λάβω υπόψιν και τα meta features που βρήκαμε παραπάνω ότι είναι αρκετά χρήσιμα. Ως καλύτερη επιλογή των features, σύμφωνα με το RFEC έχουμε:

textual_similarity,author_overlap,in_degree_j,out_degree_i,pagerank_i,pagerank_j, jaccard_similarity_neighbors,jaccard_similarity_suc_pre,word2vec_similarity, node2vec_similarity,doc2vec_similarity,cited_katz,

citing_eigenVector,cited_eigenVector,sent_transf_sim,authority_citing,authority_cited, hub_citing,hub_cited,text*outi,inj*outi, inj*doc,outi*pageJ,outi*doc,inj*hubciting,word2vec*sentence,citingK*sentence, word2vec*text, text*inj, outi*word2vec

Features	Logistic Regression
Top 10 Features	0.1554
Features Selected From RFEC	***0.1476***
All Features	0.1480
All Features + Top Engineered Features	0.1478

Εάν σε αυτά κάνουμε submit τις προβλέψεις, παίρνουμε τα παρακάτω αποτελέσματα:

Features	Logistic Regression
Top 10 Features	0.14627
Features Selected From RFEC	0.13765
All Features	0.13795
All Features + Top Engineered Features	***0.13764***

Πολύ ενδιαφέρον παρουσιάζουν αυτές οι περιπτώσεις. Και στις τέσσερις αυτό που παρατηρούμε είναι ότι το μοντέλο πηγαίνει πολύ καλύτερα στο test set από ότι όταν χρησιμοποιούμε το cross-validation.

8.3 SGD CLASSIFIER

Το επόμενο μοντέλο που χρησιμοποίησα ήταν ο SGD Classifier. Ο λόγος που χρησιμοποίησα αυτό και όχι το SVC ή το LinearSVC είναι επειδή το πρώτο μοντέλο, όπως αναφέρεται και στο documentation, είναι ακατάλληλο για περιπτώσεις όπου έχουμε αρκετά μεγάλο dataset και το δεύτερο μοντέλο δεν το χρησιμοποίησα διότι δεν είχε για loss function την log loss.

Έτσι, για να τρέξω το μοντέλο η μόνη παράμετρος που τροποποίησα ήταν του loss function να το θέσω log loss. Όπως και πριν, χρησιμοποίησα αρχικά κάποια απλά features με το cross-validation.

Features	SGD CLASSIFIER
Tf-idf sim, author overlap	0.4021
Tf-idf sim, author overlap, word2vec	0.3203

Ομοίως, θα χρησιμοποιήσω την RFEC για να βρω ποια features είναι τα καλύτερα και θα χρησιμοποιήσω πάλι τα ίδια features με το προηγούμενο μοντέλο. Με βάση το RFEC, τα καλύτερα features είναι:

'textual_similarity','author_overlap', 'in_degree_j', 'out_degree_i',

'pagerank_i', 'pagerank_j', 'jaccard_similarity_neighbors',

'jaccard_similarity_suc_pre', 'word2vec_similarity',

'doc2vec_similarity', 'sent_transf_sim', 'authority_citing',

'hub_citing', 'text*outi', 'inj*outi', 'inj*doc', 'outi*pageJ',

'outi*doc', 'word2vec*sentence', 'word2vec*text', 'outi*word2vec'

Features	SGD CLASSIFIER
Top 10 Features	0.20808
Features Selected From RFEC	***0.19081***
All Features	0.19751
All Features + Top Engineered Features	0.19120

Εάν αυτά τα κάνουμε submit, παίρνουμε τα παρακάτω αποτελέσματα:

Features	SGD CLASSIFIER
Top 10 Features	0.19974
Features Selected From RFEC	***0.18408***
All Features	0.19007
All Features + Top Engineered Features	***0.18408***

Έτσι παρατηρούμε πως τόσο με το cross validation όσο και όταν κάνουμε submit, την καλύτερη επίδοση την έχει το μοντέλο με features αυτά που επιλέγονται από το RFEC, ωστόσο την ίδια επίδοση έχει όταν γίνεται submit το τελευταίο μοντέλο με όλα τα features.

8.4 DECISION TREE CLASSIFIER

Το επόμενο μοντέλο το οποίο χρησιμοποίησα είναι το το decision_tree classifier. Επέλεξα να αφήσω όλες τις παραμέτρους στο default και να θέσω μόνο το criterion ίσο με log_loss και να τροποποιήσω και το max_depth. Αρχικά επέλεξα να θέσω την τιμή του max_depth ίση με 4 και μετά με 2.

Άρα χρησιμοποιώντας την cross_val_predict() έχω τα παρακάτω αποτελέσματα:

Max_depth	Decision Tree Classifier
2	0.33751
4	***0.21894***

Συμπεραίνουμε από τον πίνακα, ότι αποδίδει καλύτερα με max_depth να είναι ίσο με 4. Οπότε στα παρακάτω θα κρατήσω τόσο την παράμετρο. Αν εφαρμόσουμε τα ίδια features όπως κάναμε στα προηγούμενα μοντέλα, με cross_validation έχουμε:

Features	Decision Tree Classifier
Top 10 Features	0.33751
All Features	0. 33751
All Features + Top Engineered Features	***0.25858***

Αν τώρα τα κάνουμε submit, παίρνουμε τα παρακάτω αποτελέσματα:

Features	Decision Tree Classifier
Top 10 Features	0.28198
All Features	0.28198
All Features + Top Engineered Features	***0.25337***

Συμπεραίνουμε λοιπόν ότι το μοντέλο με όλα τα features και τα top engineered features αποδίδει καλύτερα. Τα πρώτα δύο τεστ παρατηρούμε ότι δίνουν ίδιο αποτέλεσμα και αυτό μπορεί να οφείλεται στο ότι όταν γίνεται η διάσπαση λαμβάνει υπόψιν τα ίδια features και αγνοεί τα επιπλέον που υπάρχουν στην 2η περίπτωση.

8.5 RANDOM FOREST CLASSIFIER

Το επόμενο μοντέλο το οποίο χρησιμοποίησα είναι το decision tree classifier. Όπως και στα Decision Trees επέλεξα να αφήσω όλες τις παραμέτρους στο default και να θέσω μόνο το criterion ίσο με log_loss και να τροποποιήσω και το max_depth. Αρχικά επέλεξα να θέσω την τιμή του max_depth ίση με 4 και μετά με 2.

Χρησιμοποιώντας λοιπόν την cross_val_predict έχουμε:

Max_depth	Random Forest Classifier
2	0.29661
4	***0.20721***

Εδώ παρατηρούμε πως αποδίδει καλύτερα με max_depth ίσο με 4. Οπότε στα παρακάτω τεστ θα συνεχίσουμε με την παράμετρο ίση με 4.

Όπως και σε όλα τα προηγούμενα μοντέλα θα εφαρμόσουμε τα ίδια features χρησιμοποιώντας την cross_val_predict.

Features	Random Forest Classifier
Top 10 Features	0.18401
All Features	0.20721
All Features + Top Engineered Features	***0.17985***

Και αν αυτά τα αποτελέσματα τα κάνουμε submit, έχουμε:

Features	Random Forest Classifier
Top 10 Features	***0.17339***
All Features	0. 19514
All Features + Top Engineered Features	0.17790

Στο συγκεκριμένο μοντέλο λοιπόν αυτό που μπορούμε να παρατηρήσουμε είναι ότι στο cross validation έχει λίγο καλύτερη επίδοση το τεστ με όλα τα features και τα engineered features ενώ στα submit αποδίδει λίγο καλύτερα το τεστ με τα 10 καλύτερα features.

8.6 MLP CLASSIFIER

Το επόμενο μοντέλο που χρησιμοποίησα είναι το MLP classifier. Επέλεξα να κρατήσω το μοντέλο σχετικά απλό, έτσι θα έχει ένα κρυμμένο επίπεδο με 16 νευρώνες και max_iter ίσο με 800. Ο λόγος που επέλεξα αυτές τις τιμές είναι επειδή δοκίμασα αρκετούς συνδυασμούς (όπως πχ 8 νευρώνες και 400 max iter, 16 νευρώνες και 1200 max iter) και κατέληξα ότι αυτός ο συνδυασμός δουλεύει καλύτερα. Ως activation function θα χρησιμοποιήσω την relu και ως solver την τεχνική adam.

Ξεκινώντας λοιπόν με την τεχνική cross validation έχουμε:

Features	MLP CLASSIFIER
Top 10 Features	0.15642
All Features	0.15219
All Features + Top Engineered Features	***0.13983***

Και αν κάνουμε submit αυτά, έχουμε:

Features	MLP CLASSIFIER
Top 10 Features	0.12620
All Features	***0.12199***
All Features + Top Engineered Features	0.14759

Έτσι βλέπουμε πως στο cross validation λειτουργεί καλύτερα το τεστ με τα 10 καλύτερα features, ενώ όταν κάνουμε submit λειτουργεί καλύτερα, με σχετικά μικρή διαφορά, το τεστ με όλα τα features.

8.7 XGBOOST

Το επόμενο μοντέλο που χρησιμοποίησα είναι το ***XGBOOST***. Αποφάσισα να εκτυπώσω και την συσχέτιση που έχουν τα features με το label καθώς και το heatmap για να δω την συσχέτιση μεταξύ τους. Προτίμησα να θέσω ως παραμέτρους το learning rate στο 0.05, το max_depth ίσο με 2 και το n_estimators ίσο με 200.

Κατά την διάρκεια του εξαμήνου που ασχολήθηκα εκτεταμένα, παρατήρησα ότι με αυτές τις παραμέτρους λειτουργεί καλύτερα.

Οπότε, όπως και σε όλες τις προηγούμενες περιπτώσεις ξεκινάμε με το cross validation:

Features	XGBoost
Top 10 Features	***0. 15152***
All Features	0.17865
All Features + Top Engineered Features	0.16135

Και αν τα κάνουμε submit, έχουμε τα εξής:

Features	XGBoost
Top 10 Features	0.14641
All Features	***0.13759***
All Features + Top Engineered Features	0.14542

Στους παραπάνω πίνακες αυτό που παρατηρούμε είναι ότι στα δύο τελευταία τεστ το cross validation δίνει πολύ υψηλότερες τιμές σε σχέση με τις πραγματικές και οι πραγματικές είναι αρκετά ικανοποιητικές.

8.8 K-NN CLASSIFIER

Το τελευταίο μοντέλο το οποίο αποφάσισα να χρησιμοποιήσω είναι το ***KNN Classifier***. Όπως φαίνεται και στην παρακάτω εικόνα, έθεσα αρχικά 5 γείτονες και μετρική της απόστασης την ***cosine***. Δυστυχώς όμως, ακόμα και με λίγα features έπαιρνε πάρα πολύ χρόνο και δεν τερμάτιζε, οπότε για το συγκεκριμένο δεν υπάρχουν αποτελέσματα.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import log_loss

X = df_pairs[["textual_similarity", "author_overlap"]]
y = df_pairs["citation_label"]

# Normalize features
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# Initializing
knn_model = KNeighborsClassifier(n_neighbors=5, metric='cosine',n_jobs=3)

#  cross-validation and probability predictions
y_proba_knn = cross_val_predict(knn_model, X_scaled, y, cv=5, method="predict_proba",n_jol

# Compute Log Loss
log_loss_value = log_loss(y, y_proba_knn)
print(f"Cross-Validation Log Loss: {log_loss_value:.5f}")
```

Εικόνα 8.4: Knn implementation

9 ΣΥΝΟΨΗ ΚΑΙ ΣΥΜΠΕΡΑΣΜΑΤΑ

Ολοκληρώνοντας λοιπόν την συγκεκριμένη εργασία, είχα την δυνατότητα να κατανοήσω σε βάθος τις τεχνικές που χρησιμοποιούνται στο NLP και να μάθω και να εκπαιδευτώ στον τρόπο με τον οποίο μπορούμε να τις εφαρμόσουμε προκειμένου να λύσουμε πραγματικά προβλήματα.

Στους παρακάτω πίνακες, μπορούμε να δούμε συγκεντρωτικά τα αποτελέσματα της εργασίας. Ο πρώτος πίνακας είναι τα καλύτερα αποτελέσματα ανά μοντέλο από το cross validation, ενώ ο δεύτερος είναι τα καλύτερα αποτελέσματα ανά μοντέλο από τα submit που συμπεριέλαβα παραπάνω.

Model	***Feature Set***	***Score (log loss)***
Gaussian Bayes	node2vec, sent_transf	0.31508
Logistic Regression	Features Selected From RFEC	0.1476
SGD Classifier	Features Selected From RFEC	0.19081
Decision Trees	All Features + Top Engineered Features	0.25858
Random Forest	All Features + Top Engineered Features	0.17985
MLP Classifier	All Features + Top Engineered Features	***0.13983***
XGBoost	Top 10 Features	0. 15152

Model	***Feature Set***	***Score (log loss)***
Gaussian Bayes	node2vec, sent_transf	0.34454
Logistic Regression	All Features + Top Engineered Features	0.13764
SGD Classifier	Features Selected From RFEC	0.18408
Decision Trees	All Features + Top Engineered Features	0.25337
Random Forest	Top 10 Features	0.17339
MLP Classifier	All Features	***0.12199***
XGBoost	All Features	0.13759